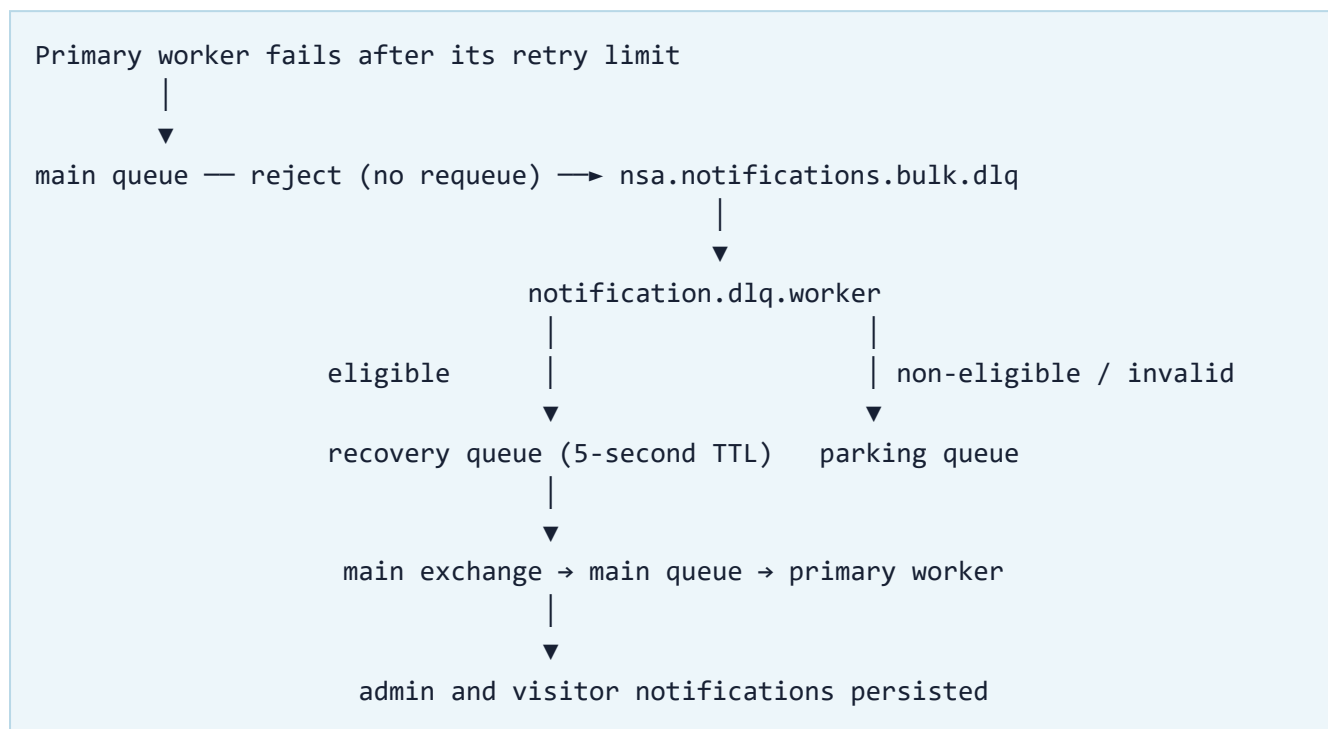


DLQ Recovery Worker

How rejected notification commands are made processable again, and where every safety decision lives in the code.



1. Where a command becomes rejected

The primary worker consumes the main queue with manual acknowledgements. When an unexpected processing failure reaches the configured application retry limit, it first records `DeadLettered` in SQL and then calls RabbitMQ with `requeue: false`. RabbitMQ routes that delivery through the configured dead-letter exchange into the DLQ.

Responsibility	Code	What it does
Main consumption	notification.worker/RabbitMqBulkNotificationWorker.cs , lines 48–56	Registers <code>AsyncEventingBasicConsumer</code> on <code>CommandQueue</code> with <code>autoAck: false</code> .
Bounded retry / final reject	notification.worker/RabbitMqBulkNotificationWorker.cs , lines 181–195	At the last attempt it calls <code>MarkDeadLetteredAsync</code> , then <code>BasicRejectAsync(..., requeue: false)</code> .
DLQ routing	notification-api/Infrastructure/Messaging/RabbitMqTopology.cs , lines 40–68	Declares the main quorum queue with <code>x-dead-letter-exchange</code> and binds <code>nsa.notifications.bulk.dlq</code> to that exchange.

2. How `notification.dlq.worker` consumes only rejected commands

The separate executable is registered in Compose as `dlq-recovery-worker`. Its hosted service opens a confirmation-tracked RabbitMQ channel, declares the shared topology, applies a prefetch of one, and consumes the DLQ with manual acknowledgement.

```
consumer.ReceivedAsync += (_, eventArgs) =>
    RecoverDeliveryAsync(channel, eventArgs, stoppingToken);

await channel.BasicConsumeAsync(
    options.Value.DeadLetterQueue,
    autoAck: false,
    consumer,
    stoppingToken);
```

Responsible code: [notification.dlq.worker/RabbitMqDeadLetterRecoveryWorker.cs](#), lines 35–71 and 101–190.

It does not replay every message found in the DLQ. Before deserializing the body, `WasRejectedFromCommandQueue` checks RabbitMQ’s `x-last-death-reason` and `x-last-death-queue` headers. It requires `rejected` and the configured main command queue. A compatible `x-death` fallback is present for broker header variations.

```
if (!WasRejectedFromCommandQueue(eventArgs.BasicProperties,
    options.Value.CommandQueue))
{
    await MoveToParkingLotAsync(..., "not-a-rejected-command-delivery", ...);
    return;
}
```

Responsible code: [notification.dlq.worker/RabbitMqDeadLetterRecoveryWorker.cs](#), lines 108–113 and 210–240.

3. How SQL is changed to `RecoveryPending`

Republishing the original body alone would not work: the main processor deliberately rejects jobs that are still marked `DeadLettered`. The recovery worker therefore invokes a small SQL workflow before publication.

```
job.Status = BulkNotificationJobStatuses.RecoveryPending;
job.QueuedAtUtc = now;
job.StartedAtUtc = null;
job.CompletedAtUtc = null;
job.Error = "Requeued by the dead-letter recovery worker.";
await jobs.SaveChangesAsync(cancellationToken);
```

Code	Behavior
notification-api/Service/Notification/BulkNotificationDeadLetterRecoveryService.cs , lines 17–50	Looks up the job. A missing job returns <code>Park</code> ; a completed job returns <code>Acknowledge</code> ; a <code>DeadLettered</code> job is durably changed to <code>RecoveryPending</code> .
notification-api/Domain/Entities/Notification/BulkNotificationJob.cs , line 14	Defines the non-terminal <code>RecoveryPending</code> status.

[notification-api/Service/Notification/BulkNotificationProcessor.cs](#), lines 42–68

Only rejects `DeadLettered` jobs; `RecoveryPending` reaches normal processing, so pending admin/visitor items are created.

Why this order matters: SQL is saved before the replay publication. If the recovery process crashes after saving but before publishing, the unacknowledged DLQ message returns and the worker can safely try again. If it publishes but cannot acknowledge the DLQ delivery, a duplicate main command is possible; the main processor acknowledges terminal jobs without replaying their completed items.

4. How the exhausted retry budget is cleared and replay is confirmed

The primary worker adds `x-retry-count` on each retry. Leaving it at 2 would immediately exhaust the new delivery's retry budget. The recovery worker clones the message properties, removes the primary retry/broker delivery headers, preserves message ID, correlation ID, type, and audit headers, then adds its own replay count.

```
headers.Remove(RetryCountHeader);
headers.Remove("x-delivery-count");
headers.Remove("x-acquired-count");
headers[ReplayCountHeader] = replayCount;
```

Responsible code: [notification.dlq.worker/RabbitMqDeadLetterRecoveryWorker.cs](#), lines 273–303.

The replay is not sent directly to the main queue. It is publish-confirmed to a durable recovery exchange/queue. That queue has a five-second TTL and dead-letters to the main exchange and main routing key. Only after `BasicPublishAsync` returns does the recovery worker acknowledge the original DLQ delivery.

```
await channel.BasicPublishAsync(
    options.Value.RecoveryExchange,
    options.Value.RecoveryRoutingKey,
    mandatory: true,
    basicProperties: replayProperties,
    body: body,
    cancellationToken: stoppingToken);
await channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false, stoppingToken);
```

Code	Behavior
notification.dlq.worker/RabbitMqDeadLetterRecoveryWorker.cs , lines 162–176	Confirmed publish first; DLQ ACK second.
notification-api/Infrastructure/Messaging/RabbitMqTopology.cs , lines 111–139	Declares the durable recovery queue, its TTL, and its dead-letter route back to the main exchange.
notification-api/Infrastructure/Messaging/RabbitMqOptions.cs , lines 42–77	Defines recovery exchange/queue names, maximum automatic replays (default 3), and five-second delay.

5. How non-replayable messages are retained safely

The worker parks a delivery instead of rejecting it from the DLQ. This is important because the DLQ itself has no downstream dead-letter route: rejecting it with `requeue: false` would discard it.

Condition	Outcome and responsible code
Not explicitly rejected from the main queue	Parked with reason <code>not-a-rejected-command-delivery</code> . RabbitMqDeadLetterRecoveryWorker.cs , lines 108–113.
Malformed JSON or unsupported message	Parked as <code>malformed-command</code> or <code>unsupported-command</code> . Lines 115–132.
Unknown SQL job	Recovery service returns <code>Park</code> ; worker parks it as <code>unknown-job</code> . Lines 147–159.
Replay limit reached	Parked as <code>replay-limit-exhausted</code> . Lines 136–145.
Recovery database/publish failure	DLQ delivery is NACKed with <code>requeue: true</code> ; if that fails, the channel closes to force broker redelivery. Lines 182–188 and 370–403.

```
await channel.BasicPublishAsync(
    options.Value.ParkingLotExchange,
    options.Value.ParkingLotRoutingKey,
    mandatory: true,
    basicProperties: parkingProperties,
    body: body,
    cancellationToken: stoppingToken);
await channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false, stoppingToken);
```

Responsible code: [notification.dlq.worker/RabbitMqDeadLetterRecoveryWorker.cs](#), lines 251–270.

The durable parking exchange/queue is declared in [notification-api/Infrastructure/Messaging/RabbitMqTopology.cs](#), lines 89–109.

6. Deployment and verification

Compose starts the new `dlq-recovery-worker` after SQL Server, RabbitMQ, and the API are healthy, then checks `/tmp/nsa-dlq-recovery-worker-ready`.

Responsible code: [notification-api/compose.week3.yml](#), lines 111–136.

Live verification performed: a two-recipient poison job reached `DeadLettered` while recovery was paused. After the primary worker's failure injection was removed and the recovery worker started, logs showed the job marked `RecoveryPending`, replayed once, and acknowledged by the primary worker. The job then reported `Completed`, with exactly one persisted notification for the admin and one for the visitor.

DLQ Recovery Worker — Refactor Update

This addendum supersedes earlier descriptions of duplicated worker-host plumbing. The replay, parking, and at-least-once delivery guarantees are unchanged.

What changed. The two deployable workers now share one small non-deployable library. Each worker contains only its queue-specific policy; the common library owns broker lifecycle, readiness, recovery from unhandled failures, and AMQP utility code.

Refactored architecture

notification.worker.shared	
Hosting/RabbitMqConsumerWorkerBase.cs	
connection + confirm-tracked channel + topology + QoS	
SQL readiness marker + reconnect loop + safe NACK fallback	
Messaging/IRabbitMqDelivery.cs and RabbitMqDelivery.cs	
publish-confirm, ACK, reject contract for one delivery	
Messaging/RabbitMqMessageHeaders.cs / RabbitMqMessageProperties.cs	
header decoding and immutable metadata copies	
Messaging/BulkNotificationCommandReader.cs	
one validation rule for the versioned command envelope	
notification.worker	notification.dlq.worker
Consumers/ thin host	Consumers/ thin host (prefetch 1)
Handlers/ command retry policy	Handlers/ replay + parking policy
	Messaging/ DLQ eligibility inspector

Code responsibility	Owner after the refactor
Connection, publisher-confirm channel, topology declaration, QoS, SQL readiness file, reconnect	RabbitMqConsumerWorkerBase
Deserializing and validating BulkNotificationRequestedV1	BulkNotificationCommandReader , shared by both policies
Command retry, SQL retry/dead-letter state, main-queue ACK/reject	BulkNotificationCommandHandler
Rejected-origin check, replay-limit check, recovery/parking publication	DeadLetterRecoveryDeliveryHandler plus DeadLetterRecoveryMessageInspector
Durable state transitions for a job	BulkNotificationProcessor and BulkNotificationDeadLetterRecoveryService

Dependency boundary improved

BulkNotificationProcessor now receives IBulkNotificationJobRepository , INotificationService , IBulkNotificationFailureInjector , TimeProvider , and its logger explicitly. It no longer constructs EF and RabbitMQ-adjacent dependencies internally. The worker composition root registers scoped business dependencies; the hosted service and delivery handlers remain singleton-safe and create one DI scope per delivery.

Who settles each message?

A handler—not the common host—owns the successful disposition of a delivery. This makes the acknowledgement order easy to audit and keeps the shared host from accidentally changing business semantics.

Main notification worker

Valid command succeeds: `BulkNotificationProcessor.ProcessAsync` persists progress and terminal state.



Then ACK: the handler acknowledges the original main-queue delivery.

Transient failure below the limit: persist `Retrying` in SQL.



Publish-confirm retry: copy the message and set `x-retry-count`.



Then ACK original: a publish failure leaves the original unacknowledged so the shared host NACKs/requeues it.

Final application attempt or a pre-existing dead-lettered job: persist `DeadLettered` when appropriate, then reject without requeue so RabbitMQ routes it to the DLQ.

The retry parser accepts only non-negative values and treats an excessive retry header as exhausted before incrementing it. This prevents an integer-overflow header from bypassing the retry budget.

DLQ recovery worker

Eligibility: accept only a message whose death metadata says `reason=rejected` from the configured main command queue. `x-last-death-*` is preferred; `x-death` is the fallback.



Prepare durable state: a matching `DeadLettered` SQL job becomes `RecoveryPending`; terminal jobs are acknowledged; unknown jobs are parked.



Publish-confirm recovery: remove `x-retry-count`, `x-delivery-count`, and `x-acquired-count`; add `x-dlq-replay-count`; publish to the durable recovery queue.



Then ACK DLQ: the recovery queue TTL routes the command back to the main queue with a fresh application retry budget.

Why the SQL update comes first. The main processor intentionally rejects a job still marked `DeadLettered`. Marking it `RecoveryPending` before the confirmed recovery publication makes the replay processable, and retrying an interrupted DLQ delivery remains idempotent.

Parking, recovery, and verification

Parking is deliberate, durable, and non-looping

Malformed commands, unsupported types/schema, unknown jobs, non-rejected DLQ messages, and replay-limit-exhausted messages are not rejected from the DLQ. Instead, the recovery handler adds `x-dlq-recovery-reason`, publish-confirms to the durable parking queue, and only then ACKs the DLQ delivery. If SQL work, replay publication, or parking publication fails, the delivery remains unacknowledged and the shared host safely NACKs/requeues it.

Question	Code to inspect
How does the DLQ consumer start and remain ready?	<code>notification.worker.shared/Hosting/RabbitMqConsumerWorkerBase.cs</code> and <code>notification.dlq.worker/Consumers/RabbitMqDeadLetterRecoveryWorker.cs</code>
Where is the recovery/parking ordering enforced?	<code>notification.dlq.worker/Handlers/DeadLetterRecoveryDeliveryHandler.cs</code>
Where are death headers and replay properties interpreted?	<code>notification.dlq.worker/Messaging/DeadLetterRecoveryMessageInspector.cs</code>
Where is the primary retry/dead-letter ordering enforced?	<code>notification.worker/Handlers/BulkNotificationCommandHandler.cs</code>
Where is recovery state made processable?	<code>notification-api/Service/Notification/BulkNotificationDeadLetterRecoveryService.cs</code>

Validation performed after this refactor

- Release build of the complete .NET solution succeeded with zero warnings/errors.
- 89 automated backend tests passed, including command-reader, retry-property, DLQ eligibility, and recovery-state coverage.
- Next.js and Angular production builds succeeded.
- The Compose API, SQL Server, RabbitMQ, primary worker, and DLQ worker were all healthy after rebuilding their images.
- A controlled two-recipient poison job reached `DeadLettered`, was recovered once, and completed with exactly one admin and one visitor notification.

Operational conclusion: the refactor improves readability and test seams without weakening the original at-least-once guarantees. The main worker still owns retries; the separate DLQ worker still owns replay and parking; the broker remains the durable transport boundary.